String handling

String Handling in Java

In Java, a **String** is a sequence of characters. Unlike many programming languages that implement strings as character arrays, Java treats strings as **immutable objects** of type String. This immutability means that once a String object is created, its contents cannot be changed. However, Java provides methods to manipulate strings efficiently.

The String Constructors: Creating Strings in Java

Java provides multiple ways to create strings:

1. **Using String Literals**

   String str = "Hello, World!";

2. **Using the new Keyword - String(char chars[ ])**

String s = new String("Hello"); //This explicitly creates a new String object in memory.

3. **Using a Character Array**

```
char chars[] = { 'J', 'a', 'v', 'a' };

String s = new String(chars);

System.out.println(s);  // Output: Java
```

4. Using a Subset of a Character Array- String(char chars[ ], int startIndex, int numChars)

```
char chars[ ] = { 'H', 'e', 'l', 'l', 'o', 'J', 'a', 'v', 'a' };

String s = new String(chars, 5, 4); // Start at index 5, take 4 characters

System.out.println(s);  // Output: Java
```

5. Copying Another String- String(String strObj)

```
String original = "Java Programming";

String copy = new String(original);

System.out.println(copy);  // Output: Java Programming
```

Example

```
class MakeString {
        public static void main(String args[]) {
                char c[ ] = { 'J', 'a', 'v', 'a' };         // Creating a string from a character array
        String s1 = new String(c);
         String s2 = new String(s1);                  // Creating a string from another string
        System.out.println(s1);                       // Output: Java
        System.out.println(s2);                      // Output: Java
        }
}
```

Example

```
class ModifyCharArray {
        public static void main(String args[]) {
        char[ ] charArray = {'H', 'e', 'l', 'l', 'o'};                  // Creating a character array
        // Modifying the first character
        charArray[0] = 'M';              // ✅ Allowed, because arrays are mutable
          System.out.println(charArray);  // Output: Mello
        }
}
```

Example:

```
class StringImmutabilityExample {
    public static void main(String args[]) {
    String s = "Hello";
// Attempting to modify a character (This will cause a compilation error)
    s[0] = 'M';  // ❌ Error: Cannot assign a value to an index in a string
    System.out.println(s);
    }
}
```

ASCII character set. Because 8-bit ASCII strings are common, the String class provides constructors that initialize a string when given a byte array. Their forms are shown here:

String(byte asciiChars[ ])

String(byte asciiChars[ ], int startIndex, int numChars)

Example:

```
class SubStringCons {
        public static void main(String args[ ]) {
        byte ascii[] = {65, 66, 67, 68, 69, 70}; // ASCII values for 'A', 'B', 'C', 'D', 'E', 'F'
        String s1 = new String(ascii);// Creating a String from the entire byte array
        System.out.println(s1);  // Output: "ABCDEF"
      String s2 = new String(ascii, 2, 3); // Creating a substring from index 2, taking 3 characters
        System.out.println(s2);  // Output: "CDE"
        }
}
```

String Length

The length of a string is the number of characters that it contains. To obtain this value, call the length( ) method, shown here:

int length( )

The following fragment prints “3”, since there are three characters in the string s:

```
char chars[ ] = { 'a', 'b', 'c' };

String s = new String(chars);

System.out.println(s.length());
```

Special String Operations in Java

Java provides built-in support for handling strings efficiently. Let's go through **string literals, concatenation, type conversion, and overriding toString()** with **examples**.

1. String Literals- Instead of manually creating a **String** object, Java allows direct assignment using **string literals**.

```
class StringLiteralExample {

        public static void main(String args[]) {

        char chars[] = { 'a', 'b', 'c' };

        String s1 = new String(chars);

        String s2 = "abc";                  // Using a string literal (simpler way)

        System.out.println(s1);  // Output: abc

        System.out.println(s2);  // Output: abc

        System.out.println("hello".length());  // Output: 5 (// Direct method call on string literal)

        }

}
```

2. String Concatenation (+ Operator)

Unlike most objects, String supports the + operator for **concatenation**.

Example: String Concatenation

```
class StringConcatExample {
    public static void main(String args[]) {
    String age = "9";
    String s = "He is " + age + " years old.";
    System.out.println(s);  // Output: He is 9 years old.
    }
}
```

3. Concatenation with Other Data Types

Java **automatically converts** non-string types to **strings** when using + with a String.

Example: Concatenating Strings with Integers

```
class ConcatWithNumbers {

    public static void main(String args[]) {

    int age = 9;

    String s = "He is " + age + " years old.";

    System.out.println(s);  // Output: He is 9 years old.

    }

}
```

Example: Operator Precedence

```
class OperatorPrecedence {
        public static void main(String args[]) {
        String s1 = "four: " + 2 + 2;  // Concatenation first
        System.out.println(s1);  // Output: four: 22

        String s2 = "four: " + (2 + 2);  // Addition first
        System.out.println(s2);  // Output: four: 4
        }
}
```

4. toString() Method- The toString() method **converts objects into strings**.

```
class Box {
    double width, height, depth;
    Box(double w, double h, double d) {
    width = w;
    height = h;
    depth = d;
    }
```

```
    public String toString() {
      return "Dimensions are " + width + " by " + depth + " by " + height + ".";
      }
}
class ToStringExample {
      public static void main(String args[]) {
      Box b = new Box(10, 12, 14);
      System.out.println(b);                    // Automatically calls b.toString()
      String s = "Box b: " + b;                 // Concatenating an object with a string
      System.out.println(s);
      }
}
```

// Overriding toString() to return a custom string representation

Output:

Dimensions are 10.0 by 14.0 by 12.0.

Box b: Dimensions are 10.0 by 14.0 by 12.0.

5. valueOf() Method- Java uses String.valueOf() to **convert primitive types to Strings**.

```
class ValueOfExample {
        public static void main(String args[]) {
        int num = 100;
        double price = 99.99;
        String numStr = String.valueOf(num);       // Converting primitive values to String
        String priceStr = String.valueOf(price);
        System.out.println(numStr);   // Output: 100
        System.out.println(priceStr); // Output: 99.99
        }
}
```

Character Extraction in Java

Java provides multiple ways to extract characters from a String. Since strings are immutable, you cannot modify them directly like character arrays, but you can retrieve individual or multiple characters using the following methods.

1. charAt(int index)

The charAt() method extracts a single character at a specified index.

Syntax:

char charAt(int index)

index → Position of the character (starts from **0**).

Returns the character at the specified position.

Example

```
class CharAtDemo {
    public static void main(String args[]) {
    String str = "Hello";
    char ch = str.charAt(1);  // Extracts 'e'
    System.out.println("Character at index 1: " + ch);
    }
}
```

2. getChars(int sourceStart, int sourceEnd, char[] target, int targetStart)

- Extracts a **substring** (multiple characters) and stores it in a **character array**.
- sourceStart → Start index (inclusive).
- sourceEnd → End index (exclusive).
- target → Destination array to store extracted characters.
- targetStart → Starting index in the target array.

Syntax:

void getChars(int sourceStart, int sourceEnd, char target[], int targetStart)

```
class GetCharsDemo {
    public static void main(String args[]) {
    String s = "Hello, World!";
    char buf[ ] = new char[5];
    // Extracts characters from index 7 to 12 (excluding 12)
    s.getChars(7, 12, buf, 0);
        System.out.println(buf);  // Prints 'World'
    }
}
```

```
class GetCharsDemo {
    public static void main(String args[]) {
        String s = "Hello, World!";
        char buf[ ] = {'g','o','o','d','-','m','o','r','n','i','n','g'};
        // Extracts characters from index 7 to 12 (excluding 12)
        s.getChars(7, 12, buf, 0);
                System.out.println(buf);  // Prints 'World'
    }
}
```

Output: **Worldmorning**

```
class GetBytesDemo {
        public static void main(String args[]) {
        String s = "ABC";
        byte[ ] bytes = s.getBytes();


             for (byte b : bytes) {
              System.out.print(b + " ");
            // ASCII values of 'A', 'B', 'C'
        }
        }
}
```

3. getBytes()

- Converts a String into a **byte array** using character encoding
- Useful when sending data over networks or writing to a binary file.

Syntax: **byte[ ] getBytes()**

4. toCharArray()

- Converts the entire String into a **character array**.

Syntax: **char[ ] toCharArray()**

```
class ToCharArrayDemo {

    public static void main(String args[]) {

    String s = "Java";

    Char[ ] chars = s.toCharArray();

    for (char ch : chars) {

        System.out.print(ch + " ");  // Prints 'J a v a'

    }    }    }
```

String Comparison in Java

Java provides multiple methods to compare `String` objects. These methods allow you to check if strings are **equal**, compare **regions** of strings, determine **lexicographic order**, and check if a string **starts or ends** with a specific substring.

1. equals() and equalsIgnoreCase()

- **equals()** → Checks if two strings are exactly the same (case-sensitive).
- **equalsIgnoreCase()** → Checks if two strings are the same, ignoring case differences.

Syntax:

```
boolean equals(Object str)            // Case-sensitive
boolean equalsIgnoreCase(String str) // Case-insensitive
```

```
class EqualsDemo {
        public static void main(String args[]) {
        String s1 = "Hello";
        String s2 = "Hello";
        String s3 = "HELLO";

        System.out.println(s1.equals(s2));          // true
        System.out.println(s1.equals(s3));          // false
        System.out.println(s1.equalsIgnoreCase(s3)); // true
        }
}
```

2. regionMatches()

Compares a specific **substring** of one string with another.

Syntax:

boolean regionMatches(int startIndex, String str2, int str2StartIndex, int numChars)

boolean regionMatches(boolean ignoreCase, int startIndex, String str2, int str2StartIndex, int numChars)

startIndex → Starting index of substring in the first string.
str2 → The second string.
str2StartIndex → Starting index in the second string.
numChars → Number of characters to compare.
ignoreCase → If true, comparison is case-insensitive.

```
class RegionMatchesDemo {
    public static void main(String args[]) {
    String str1 = "Hello, Java!";
    String str2 = "java";

    // Case-sensitive comparison
    System.out.println(str1.regionMatches(7, str2, 0, 4)); // false

    // Case-insensitive comparison
    System.out.println(str1.regionMatches(true, 7, str2, 0, 4)); // true
    }
}
```

3. startsWith() and endsWith()

- **startsWith()** → Checks if a string starts with a given prefix.
- **endsWith()** → Checks if a string ends with a given suffix.

Syntax:

boolean startsWith(String str)

boolean startsWith(String str, int startIndex) // From specific index

boolean endsWith(String str)

```
class StartsEndsDemo {
    public static void main(String args[]) {
    String str = "Programming in Java";

    System.out.println(str.startsWith("Programming")); // true
    System.out.println(str.startsWith("Java",15));          // true
    System.out.println(str.endsWith("Java"));          // true
    }
}
```

4. compareTo() and compareToIgnoreCase()

- **compareTo()** → Compares two strings lexicographically (dictionary order).
- **compareToIgnoreCase()** → Same as compareTo(), but case-insensitive.
  Syntax:

int compareTo(String str)

int compareToIgnoreCase(String str)

Returns:

- **Less than 0** → If the first string is **smaller**.
- **Greater than 0** → If the first string is **larger**.
- **Zero** → If both strings are **equal**.

```
class CompareToDemo {
        public static void main(String args[]) {
        String str1 = "Apple";
        String str2 = "Banana";
        String str3 = "apple";

        System.out.println(str1.compareTo(str2));           // -1 (Apple < Banana)
        System.out.println(str2.compareTo(str1));           // 1 (Banana > Apple)
        System.out.println(str1.compareToIgnoreCase(str3)); // 0 (Apple == apple)
        }
}
```

5. equals() vs == (Reference Comparison)

Example:

```
class EqualsVsEqualOperator {
    public static void main(String args[]) {
    String s1 = "Hello";
    String s2 = new String("Hello");  // Creates a new object
    System.out.println(s1.equals(s2));  // true (same content)
    System.out.println(s1 == s2);     // false (different memory locations)
    }
}
```

- equals() **compares values** (characters inside strings).
- == **compares references** (memory locations).

Example: Bubble Sort for Strings

```
class SortStrings {
        static String arr[] = {"Now", "is", "the", "time", "for", "all", "good", "men"};
        public static void main(String args[]) {
        for (int i = 0; i < arr.length - 1; i++) {
        for (int j = i + 1; j < arr.length; j++) {
                if (arr[i].compareTo(arr[j]) > 0) { // Swap if out of order
                String temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
                }
        }
        }
```

6. Sorting Strings Using compareTo()

The compareTo() method is often used for sorting strings in **lexicographic order**.

```
 for (String word : arr) {
            System.out.println(word);
      }
      }
}
```

Output:

Now
all
for
good
is
men
the
time

N → 78

a → 97

f → 102

g → 103

i → 105

m → 109

t → 116

t → 116

Searching Strings

The indexOf() and lastIndexOf() methods in Java are used to search for a character or a substring in a string.

Basic Usage

- indexOf(char ch): Finds the **first occurrence** of ch in the string.
- lastIndexOf(char ch): Finds the **last occurrence** of ch in the string.
- indexOf(String str): Finds the **first occurrence** of str in the string.
- lastIndexOf(String str): Finds the **last occurrence** of str in the string.

With a Starting Index

- indexOf(char ch, int startIndex): Starts searching from startIndex and finds the **first occurrence**.

- lastIndexOf(char ch, int startIndex): Starts searching **backward** from startIndex to **zero** for the last occurrence.

- indexOf(String str, int startIndex): Finds the **first occurrence** of str starting from startIndex.

- lastIndexOf(String str, int startIndex): Searches **backward** from startIndex for the **last occurrence** of str.

```
class IndexOfDemo {
    public static void main(String args[]) {
    String s = "Now is the time for all good men " +
                "to come to the aid of their country.";
    System.out.println(s.length());
    // Finding first occurrence of 't'
    System.out.println("indexOf(t) = " + s.indexOf('t'));  // 7
    // Finding last occurrence of 't'
    System.out.println("lastIndexOf(t) = " + s.lastIndexOf('t'));  // 65
```

```
// Finding first occurrence of "the"
System.out.println("indexOf(the) = " + s.indexOf("the"));  // 7
// Finding last occurrence of "the"
System.out.println("lastIndexOf(the) = " + s.lastIndexOf("the"));  // 55
// Finding 't' from index 10 onwards
System.out.println("indexOf(t, 10) = " + s.indexOf('t', 10));  // 11
// Finding 't' searching backward from index 60
System.out.println("lastIndexOf(t, 60) = " + s.lastIndexOf('t', 60));  // 55
```

```
        // Finding "the" from index 10 onwards
        System.out.println("indexOf(the, 10) = " + s.indexOf("the", 10));  // 11

        // Finding "the" searching backward from index 60
System.out.println("lastIndexOf(the, 60) = " + s.lastIndexOf("the", 60));  // 55
        }
}
```

Now is the time for all good men to come to the aid of their country.

69

indexOf(t) = 7

lastIndexOf(t) = 65

indexOf(the) = 7

lastIndexOf(the) = 55

indexOf(t, 10) = 11

lastIndexOf(t, 60) = 55

indexOf(the, 10) = 44

lastIndexOf(the, 60) = 55

Output:

Modifying a String

Because String objects are immutable, whenever you want to modify a String, you must either copy it into a StringBuffer or StringBuilder, or use one of the following String methods, which will construct a new copy of the string with your modifications complete.

1. substring() Method

The substring() method is used to extract a part of a string.

Syntax:

String substring(int startIndex)

String substring(int startIndex, int endIndex) // endIndex is exclusive

```
public class SubstringExample {
        public static void main(String args[]) {
        String str = "Hello, World!";
         // Extract substring from index 7 to the end
        String sub1 = str.substring(7);
        System.out.println(sub1);  // Output: World!
        // Extract substring from index 0 to 5 (excluding 5)
        String sub2 = str.substring(0, 5);
        System.out.println(sub2);  // Output: Hello
        }
}
```

```
public class ConcatExample {
        public static void main(String args[]) {
        String s1 = "Hello";
        String s2 = s1.concat(", World!");
        System.out.println(s2);  // Output: Hello, World!
        // Same as using +
        String s3 = s1 + ", World!";
        System.out.println(s3);  // Output: Hello, World!
        }
}
```

2. concat() Method

The concat() method appends one string to another.

String concat(String str)

3. replace() Method- The replace() method replaces all occurrences of a character or a substring.

Syntax:

String replace(char oldChar, char newChar)

String replace(CharSequence oldStr, CharSequence newStr)

```
public class ReplaceExample {
    public static void main(String args[]) {
    String str = "Java is fun!";
        // Replace character
    String replaced1 = str.replace('a', 'o');
    System.out.println(replaced1);  // Output: Jovo is fun!
    // Replace substring
    String replaced2 = str.replace("fun", "awesome");
    System.out.println(replaced2);  // Output: Java is awesome!
    }    }
```

```
public class TrimExample {
    public static void main(String args[]) {
    String str = "   Hello, Java!   ";
    System.out.println("Before trim: [" + str + "]");
    String trimmedStr = str.trim();
    System.out.println("After trim: [" + trimmedStr + "]");
    }
}
```

Output:

Before trim: [Hello, Java!]
After trim: [Hello, Java!]

4. trim() Method

The trim() method removes **leading and trailing** whitespace from a string.

Syntax: **String trim()**

In Java, the strip() method is used to remove leading and trailing whitespace from a string. It was introduced in **Java 11** and differs from trim() in that it removes all **Unicode whitespace characters**, while trim() only removes ASCII whitespace.

```
public class UnicodeExample {
    public static void main(String[] args) {
    // String with EM SPACE (\u2003)
    String text = "\u2003Hello, Java!\u2003";
    System.out.println("Original: '" + text + "'");
System.out.println("After trim(): '" + text.trim() + "'");  // trim() won't remove EM SPACE
    System.out.println("After strip(): '" + text.strip() + "'"); // strip() will remove it
    }
}
```

' ' -> Unicode: \u2003

'H' -> Unicode: \u0048

'e' -> Unicode: \u0065

...

'!' -> Unicode: \u0021

Data Conversions

valueOf() Method

The valueOf() method converts primitive data types and objects into a string.

Syntax:

static String valueOf(int num)

static String valueOf(double num)

static String valueOf(Object ob)

static String valueOf(char chars[])

static String valueOf(char chars[], int startIndex, int numChars)

```
public class ValueOfExample {
    public static void main(String args[]) {
    int num = 100;
    double price = 99.99;
    char[ ] charArray = {'H', 'e', 'l', 'l', 'o'};
    // Convert int and double to String
    String str1 = String.valueOf(num);
    String str2 = String.valueOf(price);
    // Convert char array to String
    String str3 = String.valueOf(charArray);
    // Convert a part of the char array to String
    String str4 = String.valueOf(charArray, 1, 3);  // "ell"
```

```
        System.out.println("str1: " + str1);  // Output: 100
        System.out.println("str2: " + str2);  // Output: 99.99
        System.out.println("str3: " + str3);  // Output: Hello
        System.out.println("str4: " + str4);  // Output: ell
        }
}
```

Changing the case of characters within a string

Java provides two built-in methods to change the case of characters in a String:

1. **toUpperCase()** → Converts all characters to **uppercase**.

2. **toLowerCase()** → Converts all characters to **lowercase**.

Syntax:

String toUpperCase();

String toLowerCase();

- These methods **do not modify the original string** (because strings in Java are **immutable**).
- They return a **new string** with the modified case.
- Non-alphabetic characters (like numbers and symbols) remain **unchanged**.

StringBuffer

- **Mutable** alternative to String (can be modified after creation).
- Allows **insertion, deletion, and appending** of characters.
- Automatically **expands** when more space is needed.
- Used by Java internally when performing **string concatenation (+ operator)**.

StringBuffer Constructors

1. **StringBuffer()** → Creates an empty buffer with space for **16 characters**.
2. **StringBuffer(int size)** → Creates an empty buffer with a specific **size**.
3. **StringBuffer(String str)** → Creates a buffer with **initial content** from a string and 16 extra spaces.
4. **StringBuffer(CharSequence chars)** → Creates a buffer from a **character sequence**.

Why Extra Space?

- Reduces memory **reallocation**, improving performance.
- Helps avoid **fragmentation** in memory.

length() and capacity() in StringBuffer

length() Method

- Returns the **actual number of characters** in the StringBuffer.
- Syntax: **int length()**

capacity() Method

- Returns the **total allocated storage** in the StringBuffer, including extra space for future modifications.
- Default capacity = **(string length) + 16 extra characters**.

int capacity()

```
class StringBufferDemo {
        public static void main(String args[]) {
        StringBuffer sb = new StringBuffer("Hello");


        System.out.println("Buffer = " + sb);
        System.out.println("Length = " + sb.length());    // Output: 5
        System.out.println("Capacity = " + sb.capacity()); // Output: 21 (5 + 16 extra)
        }
}
```

Output:

Buffer = Hello
Length = 5
Capacity = 21

ensureCapacity(int capacity)

- This method **ensures that the StringBuffer has a minimum specified capacity**.
- If the current capacity is **less** than the specified capacity, it increases the capacity.
- **Prevents frequent memory reallocations**, improving efficiency.

Syntax: **void ensureCapacity(int capacity)**

```
class EnsureCapacityDemo {
    public static void main(String args[]) {
    StringBuffer sb = new StringBuffer("Hello");


    System.out.println("Initial Capacity: " + sb.capacity()); // Output: 21
    sb.ensureCapacity(30);  // Ensures capacity is at least 30


    System.out.println("New Capacity: " + sb.capacity()); // Output: 42 (new capacity formula applied)
    }
}
```

new capacity = (current capacity * 2) + 2
new capacity = (21 * 2) + 2 = 42 + 2 = 42

setLength(int len)

- Used to **change the length of a StringBuffer**.
- **If the new length is greater than the current length**, null characters are added.
- **If the new length is shorter**, the extra characters are **removed**.

Syntax:

void setLength(int len)

```
class SetLengthDemo {
    public static void main(String args[]) {
    StringBuffer sb = new StringBuffer("Hello World");
    System.out.println("Before setLength: " + sb); // Output: Hello World
    sb.setLength(5);  // Trims the StringBuffer
    System.out.println("After setLength(5): " + sb); // Output: Hello
    sb.setLength(10); // Expands and fills with null characters
    System.out.println("After setLength(10): " + sb); // Output: Hello
    }
}
```

charAt(int index)

- Used to **retrieve a character** from a given index in a StringBuffer.
- The index starts from **0**.
- If the index is out of range, it throws an IndexOutOfBoundsException.

Syntax

char charAt(int index)

```
class CharAtDemo {
    public static void main(String args[]) {
    StringBuffer sb = new StringBuffer("Hello");

    // Get character at index 1 (second character)
    char ch = sb.charAt(1);

    System.out.println("Character at index 1: " + ch); // Output: e
    }
}
```

setCharAt(int index, char ch)

- Used to **modify a character** at a specific index in a StringBuffer.
- **Cannot add new characters**, only **replace** existing ones.

Syntax

void setCharAt(int index, char ch)

```
class SetCharAtDemo {
        public static void main(String args[]) {
        StringBuffer sb = new StringBuffer("Hello");
        System.out.println("Before: " + sb); // Output: Hello
        // Change 'e' to 'i' at index 1
        sb.setCharAt(1, 'i');
        System.out.println("After: " + sb); // Output: Hillo
        }
}
```

getChars(int sourceStart, int sourceEnd, char[] target, int targetStart)

- Extracts a **substring** (multiple characters) and stores it in a **character array**.
- sourceStart → Start index (inclusive).
- sourceEnd → End index (exclusive).
- target → Destination array to store extracted characters.
- targetStart → Starting index in the target array.

Syntax:

void getChars(int sourceStart, int sourceEnd, char target[], int targetStart)

```
class GetCharsDemo {
    public static void main(String args[]) {
    StringBuffer sb = new StringBuffer("Hello World");
    char buf[ ] = new char[5];
    // Extracts characters from index 7 to 12 (excluding 12)
    s.getChars(7, 12, buf, 0);
        System.out.println(buf);  // Prints 'World'
    }
}
```

```
class GetCharsDemo {
    public static void main(String args[]) {
        StringBuffer sb = new StringBuffer("Hello World");
        char buf[ ] = {'g','o','o','d','-','m','o','r','n','i','n','g'};
        // Extracts characters from index 7 to 12 (excluding 12)
        s.getChars(7, 12, buf, 0);
                System.out.println(buf);  // Prints 'World'
    }
}
```

Output: **Worldmorning**

append() Method

The append() method is used to add (or concatenate) the string representation of different types of data (such as integers, objects, strings, etc.) to the end of the StringBuffer. It has multiple overloaded versions to handle different types of data.

Syntax:

StringBuffer append(String str)

StringBuffer append(int num)

StringBuffer append(Object obj)

```
class AppendDemo {
        public static void main(String[] args) {
        String s;
        int a = 42;
     StringBuffer sb = new StringBuffer(40);  // Creating StringBuffer with initial capacity 40
        s = sb.append("a = ").append(a).append("!").toString();
        System.out.println(s);  // Output: a = 42!
        }
}
```

insert() Method

The insert() method inserts a string or other data at a specific index in the StringBuffer.

Syntax:

StringBuffer insert(int index, String str)

StringBuffer insert(int index, char ch)

StringBuffer insert(int index, Object obj)

```
class InsertDemo {
	public static void main(String[] args) {
	StringBuffer sb = new StringBuffer("I Java!");
	sb.insert(2, "like ");  // Insert "like " at index 2
	System.out.println(sb);  // Output: I like Java!
	}
}
```

reverse() Method: The reverse() method reverses the characters in the StringBuffer object.

Syntax: **StringBuffer reverse()**

```
class ReverseDemo {

    public static void main(String[] args) {

    StringBuffer sb = new StringBuffer("abcdef");

    System.out.println(sb);  // Output: abcdef

    sb.reverse();

    System.out.println(sb);  // Output: fedcba

    }

}
```

delete() and deleteCharAt() Methods

- **delete()** removes a sequence of characters from the StringBuffer.

- **deleteCharAt()** removes a single character at the specified index.

Syntax:

StringBuffer delete(int startIndex, int endIndex)

StringBuffer deleteCharAt(int loc)

```
class DeleteDemo {
    public static void main(String[] args) {
    StringBuffer sb = new StringBuffer("This is a test.");
    sb.delete(4, 7);  // Removes characters from index 4 to 6 (" is ")
    System.out.println("After delete: " + sb);  // Output: This a test.
    sb.deleteCharAt(0);  // Removes the character at index 0 ("T")
    System.out.println("After deleteCharAt: " + sb);  // Output: his a test.
    }
}
```

replace() Method

The replace() method replaces a part of the string in the StringBuffer with another string.

Syntax: **StringBuffer replace(int startIndex, int endIndex, String str)**

```
class ReplaceDemo {

    public static void main(String[] args) {

    StringBuffer sb = new StringBuffer("This is a test.");

    sb.replace(5, 7, "was");  // Replaces the substring from index 5 to 7 (" is") with "was"

    System.out.println("After replace: " + sb);  // Output: This was a test.

    }

}
```

substring() Method

The substring() method extracts a portion of the string from the StringBuffer.

Syntax:

String substring(int startIndex)

String substring(int startIndex, int endIndex)

```
class SubstringDemo {
    public static void main(String[] args) {
    StringBuffer sb = new StringBuffer("Hello, World!");
    String str1 = sb.substring(7);  // From index 7 to the end
    System.out.println("Substring: " + str1);  // Output: World!
    String str2 = sb.substring(0, 5);  // From index 0 to 4 (excluding 5)
    System.out.println("Substring: " + str2);  // Output: Hello
    }
}
```

Type Wrappers in Java

In Java, **primitive data types** (such as int, double, char, etc.) are not objects; they are simple values stored directly in memory for **performance efficiency**. However, there are situations where Java requires an **object representation** of these primitive types. This is where **Type Wrappers** come in.

What Are Type Wrappers?

Java provides **wrapper classes** in the **java.lang package** to convert primitive data types into objects. These are known as **Type Wrappers**. Each primitive type has a corresponding wrapper class:

Primitive Data Type	Wrapper Class
char	Character
byte	Byte
short	Short
long	Integer
float	Float
double	Double
boolean	Boolean

For example, an int value can be wrapped inside an Integer object like this:

int num = 10;

Integer obj = Integer.valueOf(num); // Wrapping int into an Integer object

Why Are Type Wrappers Required?

There are several reasons why Java provides type wrappers:

1. To Work with Collections and Data Structures

- Java's **Collection Framework** (ArrayList, HashMap, etc.) works only with objects, not primitive types.
- You cannot store an int directly in an ArrayList, but you can store an Integer.

Example:

```
ArrayList<Integer> numbers = new ArrayList<>();

numbers.add(5); // Auto-boxing converts int to Integer
```

2. To Use Utility Methods in Wrapper Classes

- Wrapper classes provide useful methods to work with numbers, characters, and booleans.

Example:

```
String str = "123";

int num = Integer.parseInt(str);  // Converts string to int

System.out.println(num + 10); // Output: 133
```

3. To Allow Primitive Types to Be Passed by Reference

- Java **passes primitives by value**, meaning a method cannot modify a primitive argument directly.

- Using a wrapper object allows indirect modification through object references.

```
class Test {
static void modify(Integer x) {
x = x + 10;
System.out.println("Inside method: " + x);
}
```

```
public static void main(String[] args) {
Integer num = 10;
modify(num);
System.out.println("After method call: " + num);    // Output: 10
}
}
```

```
class IntWrapper {
    int value;  // A primitive inside a wrapper object
    IntWrapper(int value) {
    this.value = value;
    }
}
public class PassByReferenceExample {
    static void modifyValue(IntWrapper num) {
    num.value += 10;  // Modifying the value inside the wrapper
    }
```

```
public static void main(String[] args) {
    IntWrapper myNumber = new IntWrapper(20);
    System.out.println("Before modification: " + myNumber.value);

    modifyValue(myNumber); // Passing object reference
    System.out.println("After modification: " + myNumber.value);
    }
}
```

4. For Autoboxing and Unboxing

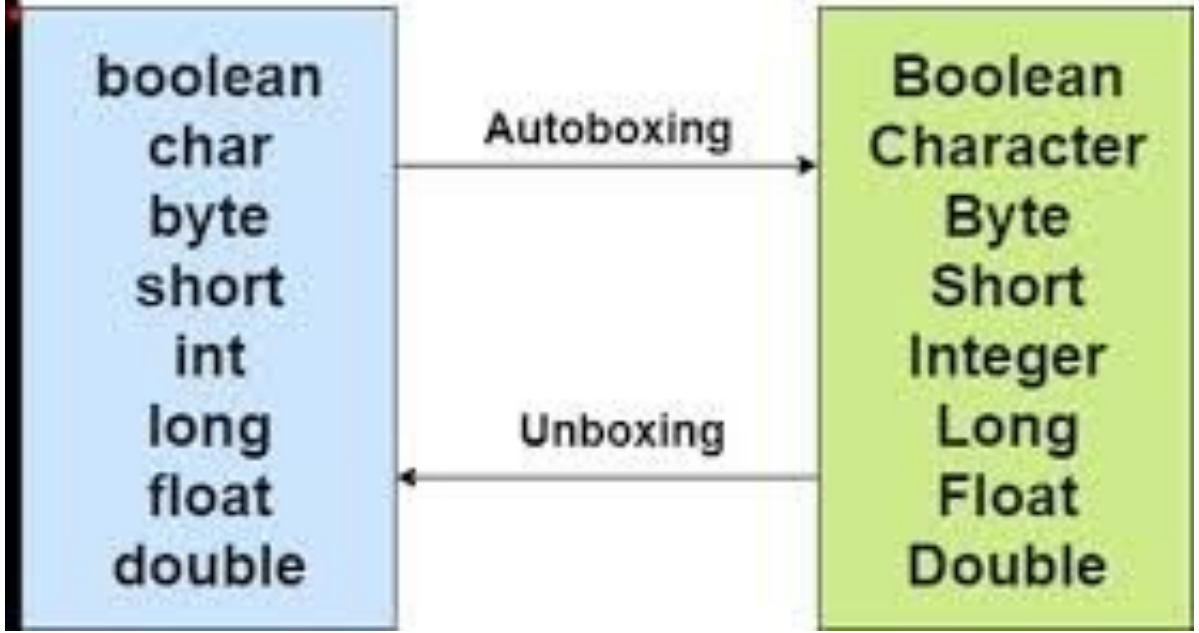
- Java **automatically** converts between primitive types and their corresponding wrapper classes through **Autoboxing** (primitive → object) and **Unboxing** (object → primitive).

Example:

```
Integer obj = 50; // Autoboxing: int → Integer

int num = obj;        // Unboxing: Integer → int
```

Wrapper Classes



1. Character Wrapper Class

The Character class wraps a char value into an object.

Syntax

Character(char ch) // Constructor to create a Character object

char charValue() // Method to extract the primitive char from the object

```
class CharacterWrapperExample {
        public static void main(String[] args) {
        Character ch = new Character('A');              // Boxing (Encapsulation)
        char c = ch.charValue();                        // Unboxing (Extracting primitive value)
        System.out.println("Character Object: " + ch);
        System.out.println("Primitive char: " + c);
        }
}
```

Output:
Character Object: A
Primitive char: A

2. Boolean Wrapper Class

The Boolean class wraps a boolean value into an object.

Syntax

Boolean(boolean boolValue) // Constructor with boolean

Boolean(String boolString) // Constructor with string ("true" or "false")

boolean booleanValue() // Extracts boolean from Boolean object

```
class BooleanWrapperExample {
        public static void main(String[] args) {
        Boolean boolObj1 = new Boolean(true);                    // Boxing
        Boolean boolObj2 = new Boolean("false");                 // String to Boolean
        boolean b1 = boolObj1.booleanValue();                     // Unboxing
        boolean b2 = boolObj2.booleanValue();                    // Unboxing
        System.out.println("Boolean Object 1: " + boolObj1);
        System.out.println("Boolean Object 2: " + boolObj2);
        System.out.println("Primitive boolean 1: " + b1);
        System.out.println("Primitive boolean 2: " + b2);
        }
}
```

Output:

```
Boolean Object 1: true
Boolean Object 2: false
Primitive boolean 1: true
Primitive boolean 2: false
```

3. Numeric Type Wrappers

Numeric wrapper classes (Byte, Short, Integer, Long, Float, Double) allow boxing and unboxing of numeric values.

Syntax

// Constructors

Integer(int num) // Creates an Integer object from int

Integer(String str) // Creates an Integer object from a string

// Methods

int intValue() // Extracts int from Integer object

double doubleValue() // Extracts double from Integer object

float floatValue() // Extracts float from Integer object

```
class NumericWrapperExample {
        public static void main(String[] args) {
        Integer numObj = new Integer(100);                      // Boxing
        int num = numObj.intValue();                            // Unboxing
        System.out.println("Integer Object: " + numObj);
        System.out.println("Primitive int: " + num);
        double d = numObj.doubleValue();                        // Unboxing
        System.out.println("Double Object: " + dObj);
        System.out.println("Primitive double: " + d);
        }
}
```

Autoboxing and Auto-unboxing in Java

Autoboxing and auto-unboxing are features introduced in **Java 5 (JDK 5)** that simplify the interaction between primitive types and their corresponding wrapper classes. These features reduce the need for explicit object creation (boxing) and extraction (unboxing) when working with primitive values and wrapper objects.

1. Autoboxing:

Autoboxing is the automatic conversion of a **primitive type** to its **wrapper class** when needed. This happens implicitly, meaning the Java compiler automatically wraps a primitive type (like int, double, char, etc.) into the corresponding object wrapper (like Integer, Double, Character, etc.) without requiring you to explicitly use the constructor.

Example: Autoboxing: primitive int to Integer

```
public class AutoBoxingExample {

    public static void main(String[] args) {

    Integer iOb = 100; // Here, int 100 is automatically boxed into an Integer object

    System.out.println("Integer object: " + iOb);

    }

}
```

2. Auto-unboxing:

Auto-unboxing is the reverse process: it is the automatic conversion of a **wrapper class** object into its **primitive type** when needed. When a primitive value is required, Java will automatically extract the value from the wrapper class object.

Example:

```
public class AutoUnboxingExample {

    public static void main(String[] args) {

    Integer iOb = 100; // Integer object

    int i = iOb; // Auto-unboxing: the Integer is automatically converted back to int

    System.out.println("Primitive int value: " + i);

    }

}
```

Autoboxing and Auto-unboxing with Methods:

Autoboxing and auto-unboxing also work when passing primitive types and wrapper objects as arguments to methods. Java automatically handles the conversion.

Example:

```
public class AutoBoxingMethodExample {

    // Method that takes Integer and returns int

    static int m(Integer v) {

    return v; // Auto-unboxing

    }

public static void main(String[] args) {

        Integer iOb = m(100); // 100 is autoboxed to Integer and passed to the method

        System.out.println("Returned Integer object: " + iOb);

        }

}
```

Autoboxing and Unboxing in Expressions:

Autoboxing and auto-unboxing can happen inside expressions too. Java automatically unboxes the wrapper objects when performing operations and reboxes the result if necessary.

Example:

```
public class AutoBoxingExpressions {

    public static void main(String[] args) {

    Integer iOb = 100;

    Integer iOb2;

    // Unboxing occurs when using iOb in the increment operation

    ++iOb; // Unboxing, incrementing, and reboxing
```

```
System.out.println("After increment: " + iOb);

    // Expression with auto-unboxing and auto-boxing

    iOb2 = iOb + (iOb / 3); // Unbox, evaluate expression, rebox result

    System.out.println("iOb2 after expression: " + iOb2);

    // Direct unboxing and operation on a primitive type

    int i = iOb + (iOb / 3); // Unboxing iOb to int, evaluating, and assigning to primitive int

    System.out.println("i after expression: " + i);

    }

}
```

Output:

After increment: 101

iOb2 after expression: 134

i after expression: 134

Using Autoboxing with Boolean and Character Values:

Java also supports autoboxing and auto-unboxing for Boolean and Character wrapper classes.

Example:

```
public class AutoBoxingBooleanCharacter {

    public static void main(String[] args) {

    // Autoboxing and auto-unboxing for Boolean

    Boolean b = true; // Autoboxing a boolean value

    if (b) {

     System.out.println("b is true"); // Auto-unboxing b to boolean for the conditional
```

```
    }
    // Autoboxing and auto-unboxing for Character
    Character ch = 'x'; // Autoboxing a char value
    char ch2 = ch; // Auto-unboxing a Character object to char
    System.out.println("ch2 is " + ch2);
    }
}
```